

Task Prompt Template

Purpose: the first message of a new conversation, every time you start a task. One conversation per task ID — never bundle two task IDs into one conversation, and never carry an old conversation's context into a new task by continuing it. A fresh agent has zero memory of anything said before this message, so this template exists to front-load exactly what it needs and nothing it doesn't.

Why this matters: [PLANNING_REVIEW.md](#) and the CI architecture-guard job (`scripts/ci/architecture_guard.py`) both exist because "the agent should have known that from the architecture doc" is not a real safety net — an agent that wasn't told a constraint in its own context window will not reliably re-derive it. This template is the primary defense; CI is the backstop that catches what the template missed.

How to use this

1. Copy the template below into a new conversation as your **first message**.
 2. Fill in every bracketed field. Do not leave [TBD] on anything you can look up yourself (task table row, file paths) — only leave it blank if the answer requires a decision only you can make (e.g., a secret value, a business rule not yet in the docs).
 3. Paste the relevant task row directly from [DELIVERABLE_1_SYSTEM_DESIGN.md](#) — don't summarize it from memory, copy the actual row so the acceptance criteria and dependency list are exact.
 4. If the task touches security, licensing, tenant isolation, or payments, also paste the relevant section of [ARCHITECTURE_SECURITY.md](#) — don't just link it. A linked-but-unread doc is not context; a pasted section is.
 5. One task ID per conversation. If you catch yourself wanting to ask for "and also do P1-T09 while we're here" — stop, finish this conversation, start a new one.
-

Template

```
# Task: [P#-T##] <Task Name, copied exactly from DELIVERABLE_1_SYSTEM_DESIGN.md>

## Task Table Row (paste verbatim – do not paraphrase)
<paste the exact markdown table row for this task from DELIVERABLE_1_SYSTEM_DESIGN.md,
including Description, Assigned Dev, Dependencies, and Est. Days>

## Phase & Position
- **Phase**: [N] – [Phase Name]
- **Dependencies already done**: [list dependency task IDs and confirm each is merged
to `develop`, e.g. "P1-T07 merged in PR #23, P1-T09 merged in PR #31"]
- **Blocks**: [list any task IDs that depend on this one, if known – helps the agent
understand what it must not break for downstream work]

## Relevant Architecture Constraints (paste sections, don't just link)
<Paste the specific paragraph(s) from ARCHITECTURE_SECURITY.md,
ARCHITECTURE_DATA_PLATFORM.md, or the DELIVERABLE_1 "Collaboration Workflow"
rules (§7) that apply to THIS task. Examples:
  - Touching licensing/menus? Paste ARCHITECTURE_SECURITY.md §4 (Defense in Depth, Ring 1/2/3)
  - Touching payments/webhooks? Paste §11 (Platform-Layer Specific Risks) webhook row
  - Touching tenant DB routing? Paste ARCHITECTURE_DATA_PLATFORM.md §3/§4
  - Cross-layer (platform addon touching tenant models)? State the two-layer rule explicitly
If nothing in the architecture docs applies beyond the general coding rules, say so
explicitly: "No task-specific architecture constraint beyond the standard rules below."
>

## Files In Scope
- **Primary addon**: `custom_addons/[addon_name]/`
- **Files this task will likely touch**: [list specific paths if known, e.g.
`models/tenant.py`, `security/ir.model.access.csv`, `views/tenant_views.xml`]
- **Files this task must NOT touch**: [call out anything adjacent that's out of scope –
e.g. "do not modify ncollection_saas – this task is ncollection_subscription only"]

## Environment & Access
- **Repo**: /Users/omaressam/Documents/ERP_Sys/ncollection-erp (or the actual path in this ses:
- **Branch**: create `feature/[task-id]-short-description` off `develop`
- **Local stack**: `docker compose up` – Odoo on :8069, Postgres via the `db` service
(see `docker-compose.yml`; dev credentials are `odoo`/`odoo`, never used in prod)
- **Secrets needed for this task**: [list only if the task genuinely needs one, e.g.
"Stripe test-mode webhook signing secret – ask before starting, do not stub with
a fake value that could accidentally leak into a commit"]
- **No secret should ever be pasted into this prompt or committed.** If a task needs a
real credential, it comes from `.env` (gitignored) or a secrets manager reference –
never inline.

## Standing Rules (apply to every task – copied here so the agent doesn't have to
## go hunting; keep this block verbatim across all prompts)
1. Odoo 19 view syntax only: `

` not ``, no `attrs=`.
2. No Odoo core files modified. Ever.
3. Two-layer separation: platform-layer addons (`ncollection_saas`,
`ncollection_subscription`) never directly query tenant ERP models
(`sale.order`, `stock.move`, `account.move`, etc.) – cross-layer access goes
through RPC/JSON-RPC only.
4. Any UI-level restriction (menu hiding, `groups=`) must have a matching
ORM/RPC-layer enforcement (`ir.rule`, `check_access_rights` override) in the
same PR if the restriction is licensing- or security-relevant. UI hiding alone
is not security.
5. OCA-first: before writing custom code, check whether an OCA module already
solves this. Record the decision (used OCA module X / built custom because Y)
in the PR description.
6. Small, incremental commits. Never generate an entire module in one shot –
```

- build it the way a human would: model → security → views → tests.
7. Every new model/field that touches money, PII, or tenant identity gets a test.
 8. Before finishing: run ``flake8 custom_addons/`` and ``python scripts/ci/architecture_guard.py --base origin/develop`` locally – fix anything they catch before opening the PR, don't rely on CI to find it first.

Definition of Done (for this specific task)

<Paste the Acceptance criteria from the task table row here again, as an explicit checklist – repetition from the row above is intentional, it's the part most likely to get lost if only stated once.>

- [] <acceptance criterion 1>
- [] <acceptance criterion 2>
- [] ...
- [] ``flake8`` clean
- [] ``architecture_guard.py`` clean (or violations justified in PR description)
- [] Tests added/updated and passing locally
- [] PR opened against ``develop``, title ``[P#-T##] <Task Name>``, description references this task ID and links any relevant doc sections used

What NOT to do

- Do not start work on any other task ID in this conversation.
- Do not modify files outside "Files In Scope" without stopping to explain why first.
- Do not invent acceptance criteria beyond what's in the task table – if something seems missing, flag it as a question rather than assuming.
- Do not mark the task done without running the two local checks in rule 8.

Worked Example

Here's the template filled in for a real task, to show the level of detail expected:

Task: [P1-T10] License Enforcement at ORM & RPC Layer

Task Table Row

| ****P1-T10**** | License Enforcement at ORM & RPC Layer | Menu hiding is a UI convenience, NOT security – a savvy user can hit unlicensed models via direct URLs or XML-RPC/JSON-RPC. Enforce licensing at the data layer: (1) map licensed module set → allowed model namespaces, (2) generate/activate `ir.rule` record rules and/or override `check\_access\_rights` via an AbstractModel mixin so read/write/create/unlink on models of unlicensed modules is denied for all non-system users, (3) return a branded "not in your plan" error (upsell message) instead of a raw AccessError where the UI can catch it, (4) automated tests: RPC calls against unlicensed models must be denied for a Starter tenant and allowed for Enterprise, (5) measure ORM overhead – enforcement must add < 5ms per request. | `[DEV-2]` | P1-T09 | 3 |

Phase & Position

- ****Phase****: 1 – Customer Workspace
- ****Dependencies already done****: P1-T09 (Module Visibility Engine / menu hiding) merged to `develop` in PR #31.
- ****Blocks****: P1-T21 (Phase 1 Integration Testing & Security Audit) explicitly tests this task's enforcement – do not consider this task done until its RPC-bypass tests would pass under P1-T21's attack scenarios.

Relevant Architecture Constraints

Pasted from ARCHITECTURE_SECURITY.md §4 (Defense in Depth):
"Ring 1 (menu hiding, P1-T09) is a UI convenience only. Ring 2 (this task) is the actual security boundary – every model belonging to an unlicensed module must deny read/write/create/unlink for non-system users regardless of access path (UI, direct URL, XML-RPC, JSON-RPC). Ring 3 (non-installation) is the provisioning-time default-deny for modules never installed on a tenant DB at all." Overhead budget: <5ms/request, verified in P3-T03 load testing – do not implement a check that requires a network call or heavy query per request.

Files In Scope

- ****Primary addon****: `custom\_addons/ncollection\_core/`
- ****Files this task will likely touch****: `models/license\_enforcement\_mixin.py` (new), `security/ir.model.access.csv`, `tests/test\_license\_enforcement.py` (new)
- ****Files this task must NOT touch****: `ncollection\_subscription` (models already merged in P1-T07 – read-only reference, don't modify)

Environment & Access

- Repo: /Users/omaressam/Documents/ERP_Sys/ncollection-erp
- Branch: `feature/p1-t10-orm-license-enforcement` off `develop`
- Local stack: `docker compose up`, two test tenant DBs already exist locally (`clienta` = Starter plan, `clientb` = Enterprise – see P1-T06 setup notes)
- Secrets needed: none

Standing Rules

[... verbatim block from the template above ...]

Definition of Done

- [] RPC calls against unlicensed models denied for Starter tenant (`clienta`)
- [] Same calls allowed for Enterprise tenant (`clientb`)
- [] Branded "not in your plan" error surfaces in UI instead of raw AccessError
- [] Measured overhead < 5ms/request (include the measurement method in the PR)
- [] flake8 clean
- [] architecture_guard.py clean
- [] Tests added and passing locally
- [] PR opened against `develop`, title `[P1-T10] License Enforcement at ORM & RPC Layer`

Notes on the "new conversation per task" question

Yes — one conversation per task ID is correct, and it's what this template is built around. A few refinements worth knowing:

- **When to break this rule:** a task that's genuinely too small to justify the template overhead (e.g., a 1-line config fix) can piggyback on an adjacent conversation if you're already there — but anything with its own task ID in the plan should get its own conversation. If it's in the table, it gets the template.
- **Long-running tasks:** a 5-day task (the maximum size in this plan) may span multiple work sessions. That's fine — it's still one conversation, resumed, not a new one each session, since resuming preserves the context this template established. Only start a *new* conversation when moving to a *different* task ID.
- **Follow-up bugs on a task already merged:** if QA finds a bug in P1-T10 after merge, that's a new, smaller prompt (same template, but Task Table Row becomes "bugfix following P1-T10, see PR #40" and Definition of Done is just the bug's reproduction case) — not a reopened P1-T10 conversation.